JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

Project Title : JSON Refiner Advanced Edition

Student name :

1. JANA SRI M
2. ALICE SAHAYA ANBARASI S
3. GANGADEVI M
4. T.Rubini
5. VIJAYESHWARI P

Register Number:

1. 23129ER014
2. 23129ER002
3. 23129ER009
4. 23129ER018
5. 23129ER024

Institution: SRI ADI CHUNCHANAGIRI WOMEN'S COLLEGE

Department: B.Sc., INFORMATION TECHNOLOGY

Date of Submission: 08/03/2026

Github Repository Link: https://github.com/jana25-mani/NM---JSON-Refiner-Advanced-Edition.git

JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

Project Description

JSON Refiner Advanced Edition is an intelligent JSON processing platform that leverages Python's advanced data handling capabilities to provide comprehensive JSON parsing, validation, transformation, and refinement with professional styling through Gradio's interactive interface. The platform addresses the challenge of JSON data quality and format standardization by delivering AI-powered JSON generation, automatic type inference, schema validation, case conversion, flattening/unflattening, and production-ready output.

Using sophisticated parsing algorithms and Python's type system, the system analyzes unstructured key-value data, automatically infers correct data types (strings, numbers, booleans, nulls, arrays, objects), applies consistent naming conventions (snake_case,camelCase, kebab-case, PascalCase), validates against JSON schemas, and provides multiple transformation options. The system ensures professional output through Gradio's user-friendly interface while maintaining data integrity through comprehensive validation and error reporting.

JSON Refiner Advanced transforms JSON processing into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and advanced algorithms that handle complex transformations while maintaining data consistency, accessibility standards, and production-ready code quality.

Scenarios

Scenario 1: Data Migration from CSV to JSON

A data analyst needs to convert a flat CSV file with mixed data types into structured JSON format. The system receives key-value pairs from the CSV (name: John, age: 28, active:
true, score: 95.5), automatically detects the correct data types, applies consistent naming conventions, and generates clean, production-ready JSON. The analyst can validate against a predefined schema to ensure data quality before migration to their database system.

Scenario 2: API Response Normalization

A backend developer receives JSON responses from multiple third-party APIs with inconsistent key naming conventions and data structures. The system standardizes all keys to a consistent case style (e.g., converting FirstName, first_name, firstName to a single convention), merges multiple API responses into a unified structure, removes null/empty values, validates against a master schema, and produces clean data for frontend consumption. This eliminates the need for manual data transformation in application logic.

Scenario 3: Configuration File Management

A DevOps engineer manages configuration files in simple text format and needs to convert them to JSON for automated deployment systems. The system parses the configuration (database_host: localhost, port: 5432, ssl_enabled: true, retry_count: 3), infers types correctly, validates required fields, prevents duplicate keys with detailed error reporting, and generates valid JSON ready for infrastructure-as-code tools. The engineer can maintain human-readable configs while ensuring JSON compliance.

Scenario 4: Data Flattening for Analytics

An analytics team has deeply nested JSON structures from their application and needs to flatten them for spreadsheet analysis. The system converts nested objects like user: {name: "John", profile: {age: 28}} to flat dot-notation (user.name: "John", user.profile.age: 28), making the data compatible with CSV/Excel tools. Later, they can unflatten the data back to nested structure when needed, maintaining complete reversibility.

Scenario 5: Schema-Based Data Validation

A quality assurance team validates JSON data against company standards using JSON Schema. The system accepts raw data, applies transformations, validates structure and field requirements, provides detailed error messages for missing/invalid fields, and prevents invalid data from entering the pipeline. The comprehensive validation ensures data consistency across all systems and catches errors early.

Architecture Overview

JSON Refiner Advanced is built as a modular platform combining Python backend with sophisticated JSON processing algorithms for intelligent data transformation.

The architecture prioritizes data integrity, user experience, and production readiness through Gradio's interactive interface and comprehensive validation mechanisms.

Core Technologies

- Python 3.8+: Core programming language for robust data processing
- Gradio 4.0+: Interactive web interface framework for user interaction
- JSON5: Advanced JSON parsing for flexible input handling
- JSONSchema: Schema validation and data conformance checking
- Regular Expressions: Pattern matching for advanced parsing
- Google Colab: Cloud-based development and deployment environment
- HTML5/CSS3: Modern web standards for interface rendering

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- pip: Python package manager (comes with Python)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and Gradio sharing

Knowledge Prerequisites

- Python Basics: Functions, dictionaries, list comprehensions, string manipulation
- JSON Fundamentals: JSON structure, types, and validation
- Regular Expressions: Pattern matching for data parsing
- Gradio Framework: Understanding of interface components and callbacks
- Data Validation: Familiarity with JSON Schema and validation concepts
- Google Colab: Basic notebook interface and cell execution

Hardware Requirements

- Processor: Any modern processor (Intel i3 or equivalent)
- RAM: Minimum 2GB local (Colab provides cloud resources)
- Storage: 500MB free space for application and generated files
- Display: Minimum 1024x768 resolution for interface usability.

Project Workflow

Phase 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies & Configure Environment

- Create requirements.txt with all dependencies
- Set up Google Colab environment with proper settings
- Install required packages: gradio, json5, jsonschema, python-dotenv, pydantic
- Verify all packages are installed correctly

Activity 1.2: Design Core Data Structures

- Define CaseStyle Enum for case conversion options
- Create JSONRefinerCore class with all processing methods
- Design type inference system for automatic data type detection
- Implement error tracking and logging mechanisms

Phase 2: Core JSON Processing Engine

Activity 2.1: Implement Type Inference System

- Create infer_type() method for automatic type detection
- Handle boolean conversions (true/false, yes/no, on/off)
- Implement null value handling (null, none, nil, n/a)
- Parse numbers (integers and floats)
- Process arrays and objects from JSON strings
- Default to string for unrecognized values

Activity 2.2: Build Key Normalization & Case Conversion

- Implement to_snake_case() conversion
- Implement to_camelCase() conversion
- Implement to_kebab_case() conversion
- Implement to_pascal_case() conversion
- Create normalize_key() method that applies selected case style

Phase 3: Validation & Transformation Features

Activity 3.1: Implement JSON Schema Validation

- Create validate_json_schema() method
- Use jsonschema library for Draft7 validation
- Implement error message formatting
- Test against various schema definitions

Activity 3.2: Build Required Fields Checking

- Create check_required_fields() method
- Implement field presence validation
- Generate missing field reports
- Combine with schema validation results

Activity 3.3: Develop JSON Transformation Methods

- Implement flatten_json() for nested to flat conversion
- Implement unflatten_json() for dot-notation reversal
- Create merge_json_objects() for deep merging
- Implement remove_null_values() for data cleanup
- Test all transformation methods with complex data

Phase 4: Gradio Interface Development Activity

4.1: Design Core Refining Tab

- Create input fields for key-value text
- Add case style radio button selector
- Implement remove nulls and flatten checkboxes
- Build JSON output code display
- Add status and statistics output panels

Phase 5: Integration, Testing & Deployment

Activity 5.1: Google Colab Deployment

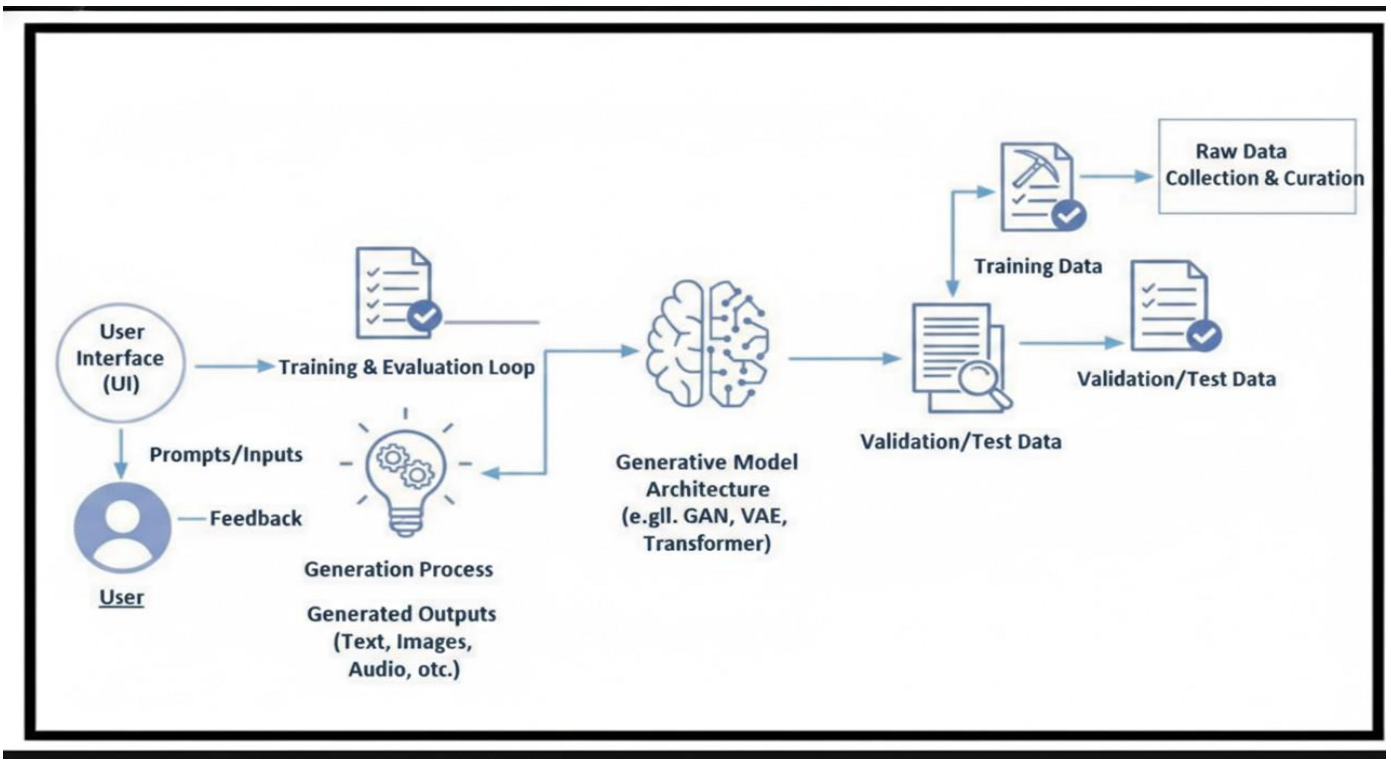
- Connect all interface tabs
- Test data flow between components
- Verify transformation pipeline
- Test error handling

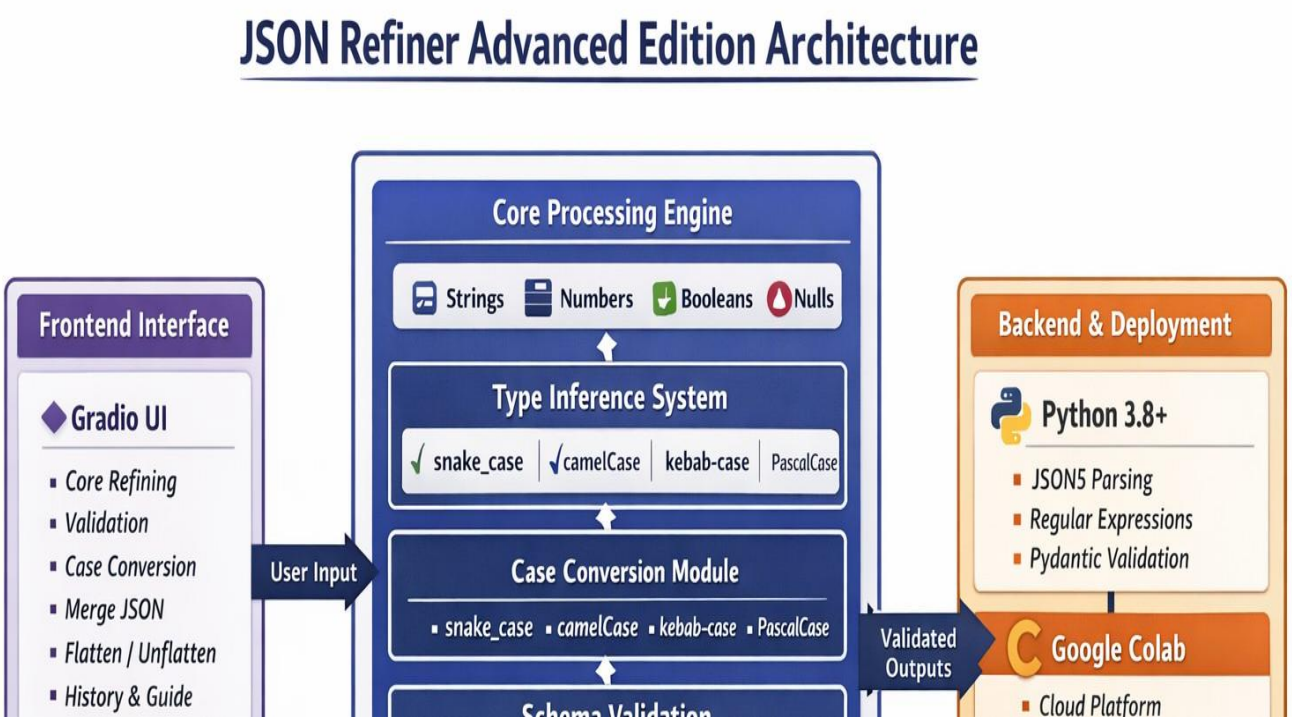
Activity 5.2: Comprehensive Testing

- Test all case conversion styles
- Validate type inference accuracy
- Test schema validation with various schemas
- Test merging with conflicting keys
- Test flattening with deeply nested objects

Technical Architecture

Project Structure

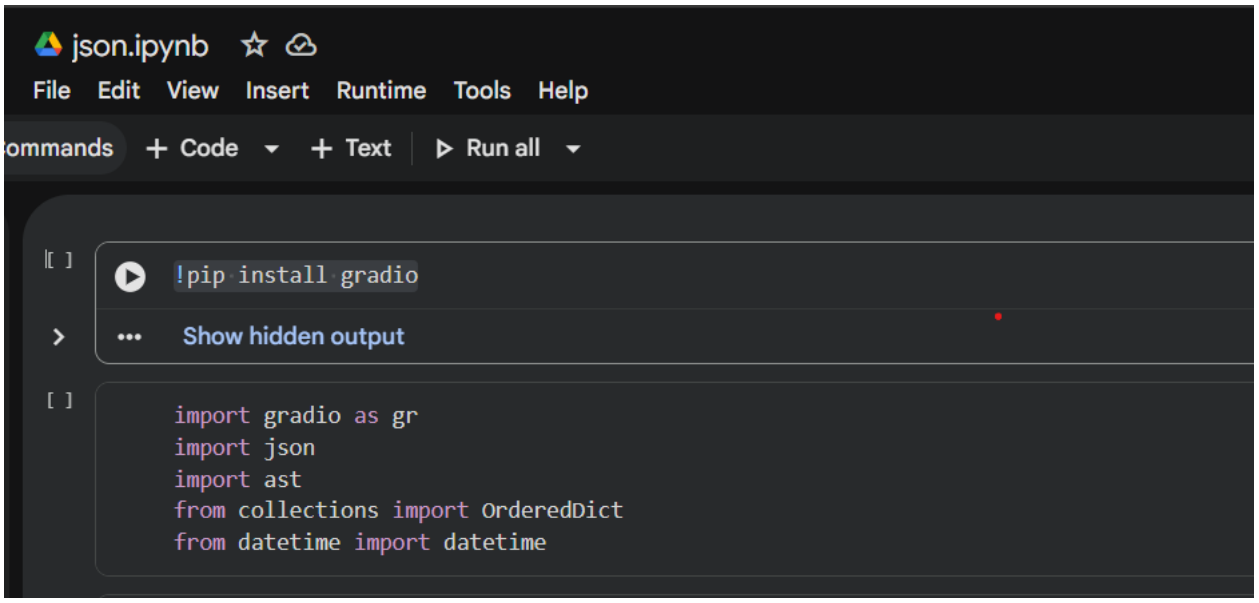




MILESTONE 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies

Load the model :



Create requirements.txt with the following dependencies:

```
[ ]
history = []

[ ]
def parse_value(value):

    value = value.strip()

    if value.lower() == "true":
        return True

    if value.lower() == "false":
        return False

    if value.lower() == "null":
        return None

    try:
        if "." in value:
            return float(value)
        return int(value)
    except:
        pass

    try:
        return ast.literal_eval(value)
    except:
        pass

    return value
```

- gradio==4.0+
- json5==0.9.6+
- jsonschema==4.17.0+
- python-dotenv==0.19.0+
- pydantic==1.9.0+

```
[ ]
def convert_case(key, style):

    if style == "snake_case":
        return key.lower().replace(" ", "_")

    if style == "camelCase":
        parts = key.split("_")
        return parts[0] + ''.join(word.capitalize() for word in parts[1:])

    if style == "PascalCase":
        return ''.join(word.capitalize() for word in key.split("_"))

    if style == "kebab-case":
        return key.lower().replace("_","-")

    return key
```

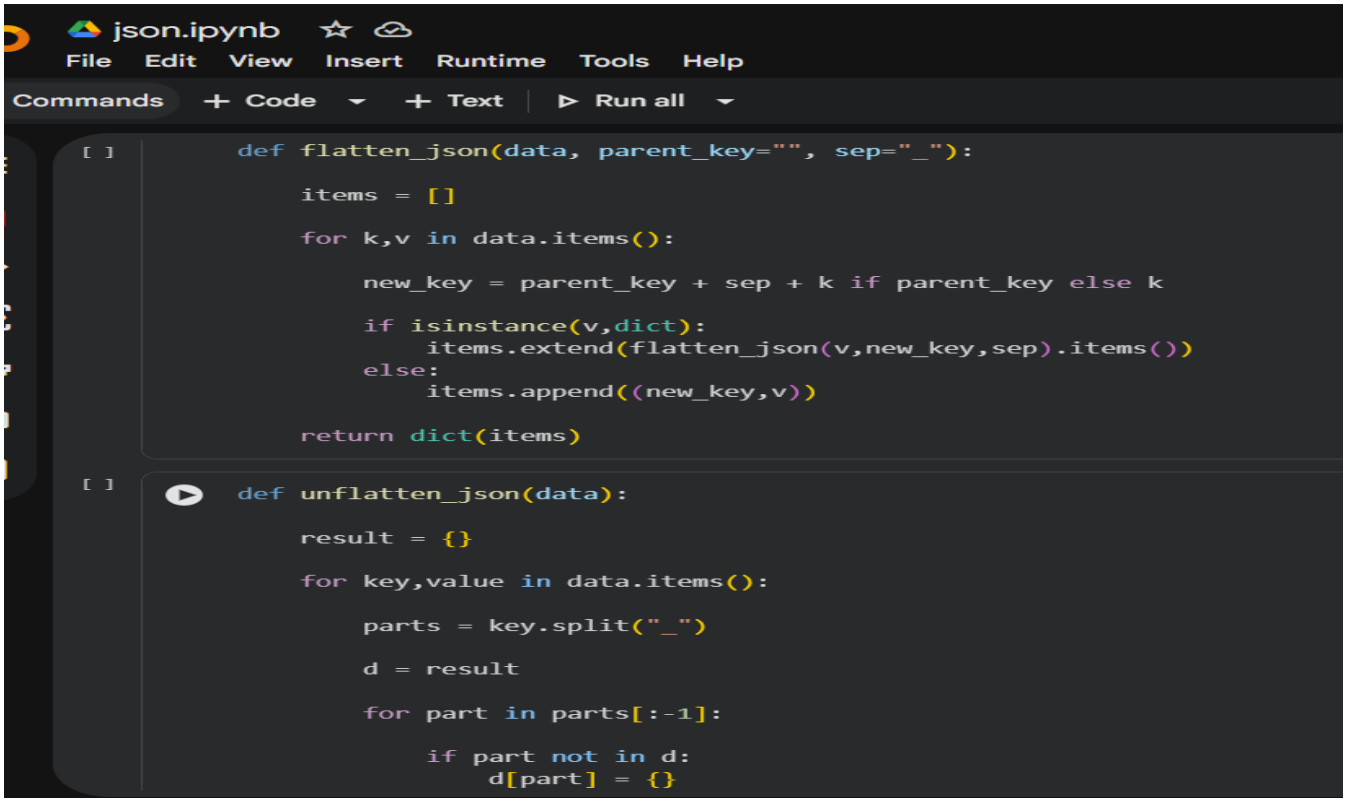
Activity 1.2: Initialize Enum and Core Class

Create CaseStyle Enum:

class CaseStyle(Enum):

SNAKE_CASE = "snake_case" CAMEL_CASE = "camelCase"

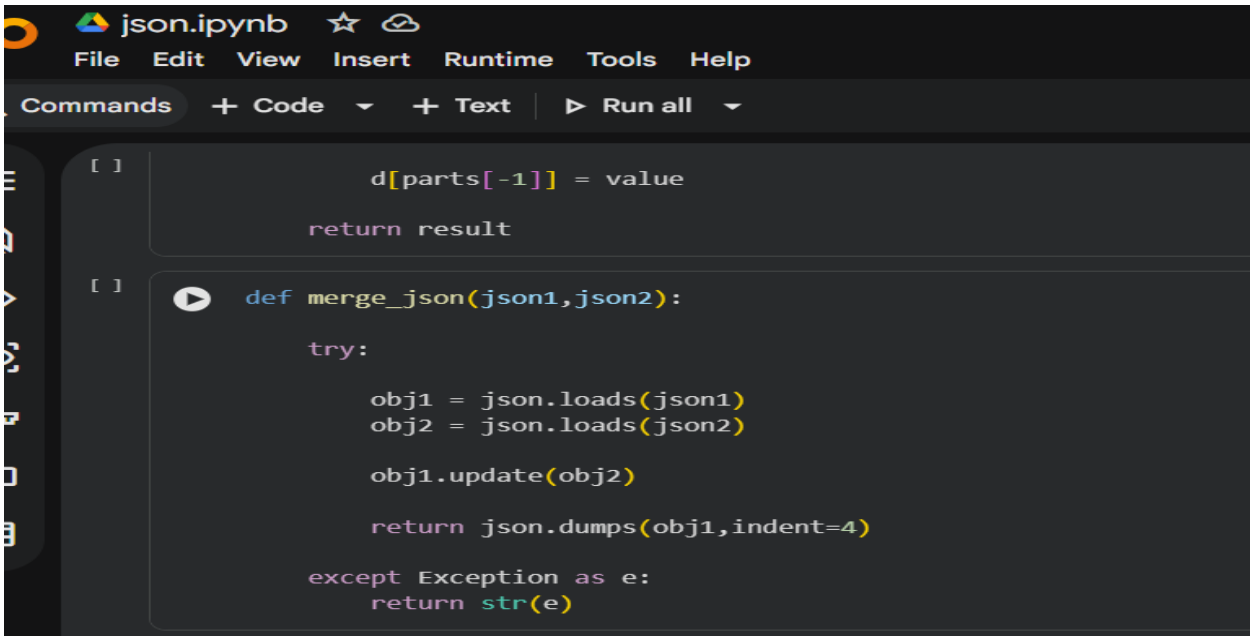
KEBAB_CASE = "kebab-case" PASCAL_CASE = "PascalCase"



Initialize JSONRefinerCore class with tracking: class JSONRefinerCore:

def __init__(self):

self.conversion_log = [] self.validation_errors = [self.transformation_history = []



MILESTONE 2: Type Inference & Case Conversion System

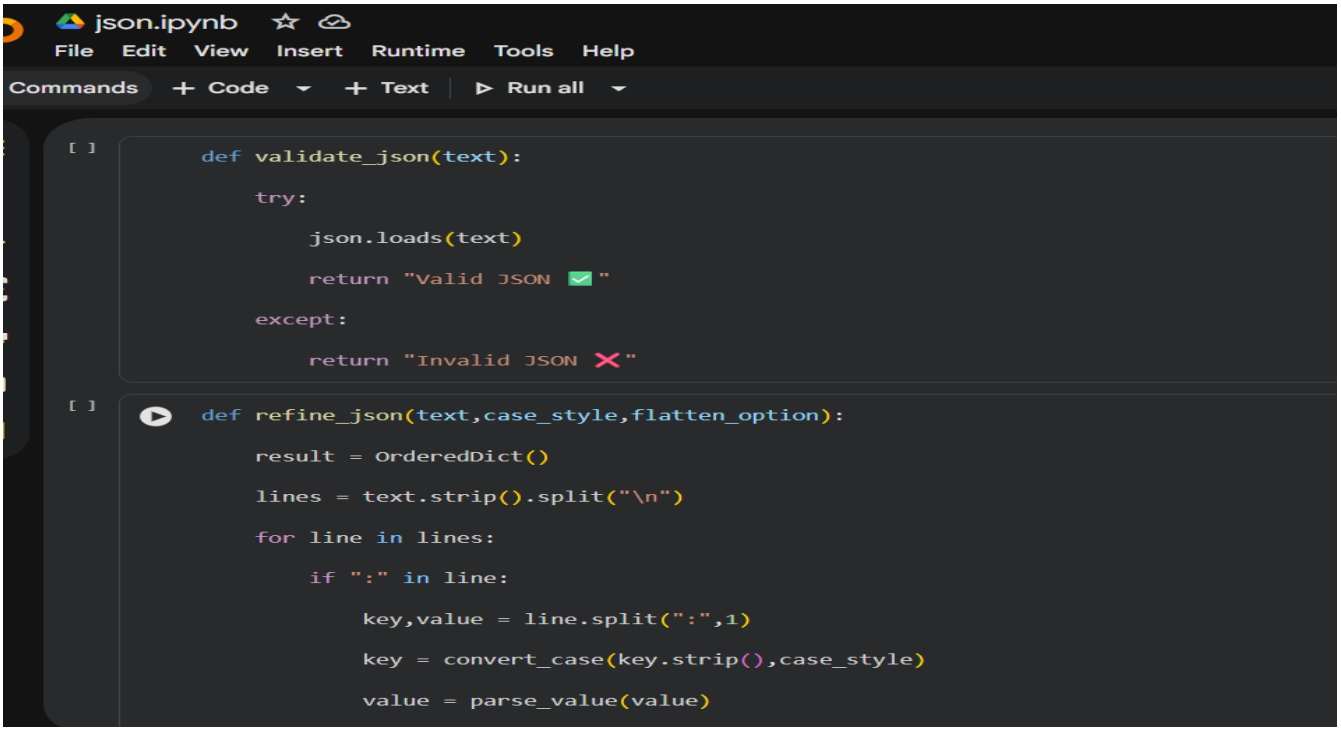
Activity 2.1: Implement Type Inference

The type inference system automatically detects and converts values:

- Booleans: Recognizes true, false, yes, no, on, off, 1, 0
- Nulls: Handles null, none, nil, undefined, n/a
- Numbers: Detects integers and floating-point numbers
- Arrays: Parses JSON arrays from string format
- Objects: Parses JSON objects from string format
- Strings: Default fallback for unrecognized values

Example: "age: 28" → {"age": 28} (integer)

Example: "active: true" → {"active": true} (boolean)



Activity 2.2: Build Case Conversion Methods

Implement four case conversion standards:

1. snake_case: user_name, first_name, is_active
2. camelCase: userName, firstName, isActive
3. kebab-case: user-name, first-name, is-active
4. PascalCase: UserName, FirstName, IsActive

Each method uses regex patterns to normalize input:

- Remove non-alphanumeric characters
- Split on boundaries (spaces, underscores, hyphens)
- Apply case transformation
- Validate output format

```
                result[key] = value

        if flatten_option:

            result = flatten_json(result)

        output = json.dumps(result,indent=4)

        history.append({
            "time":str(datetime.now()),
            "output":output
        })

        return output
```

```
def show_history():

    text=""

    for h in history:

        text += f"\nTime : {h['time']}\n"
        text += f"{h['output']}\n"
        text += "-----------------\n"

    return text
```

MILESTONE 3: Validation & Transformation Features

Activity 3.1: JSON Parsing & Validation

Key-value parsing system:

```
guide_text = """
Input Format Example

name: John Doe
age: 28
is_active: true
balance: 1500.50

Features

Core Refining
Validation
Case Conversion
Flatten JSON
Merge JSON
History Tracking
"""
```

Activity 3.2: Schema & Field Validation

Dual validation approach:

1. Required Fields: Check for mandatory fields

   - Input: "name, age, email"

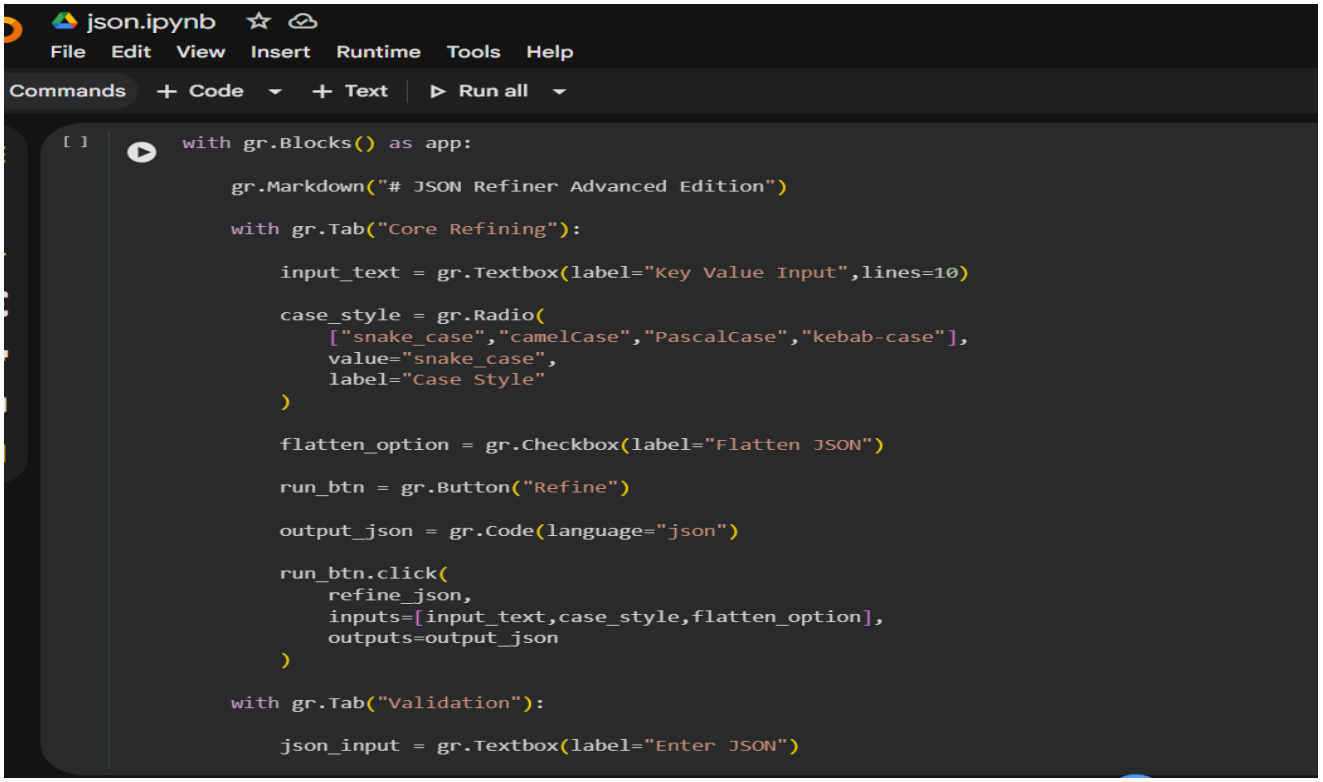
   - Result: Lists missing fields

2. Schema Validation: Validate against JSON Schema

   - Input: {"type": "object", "properties": {...}}

   - Result: Full structural validation

   - Supports Draft 7 standard




```
with gr.Blocks() as app:

    gr.Markdown("# JSON Refiner Advanced Edition")

    with gr.Tab("Core Refining"):

        input_text = gr.Textbox(label="Key Value Input",lines=10)

        case_style = gr.Radio(
            ["snake_case","camelCase","PascalCase","kebab-case"],
            value="snake_case",
            label="Case Style"
        )

        flatten_option = gr.Checkbox(label="Flatten JSON")

        run_btn = gr.Button("Refine")

        output_json = gr.Code(language="json")

        run_btn.click(
            refine_json,
            inputs=[input_text,case_style,flatten_option],
            outputs=output_json
        )

    with gr.Tab("Validation"):

        json_input = gr.Textbox(label="Enter JSON")
```

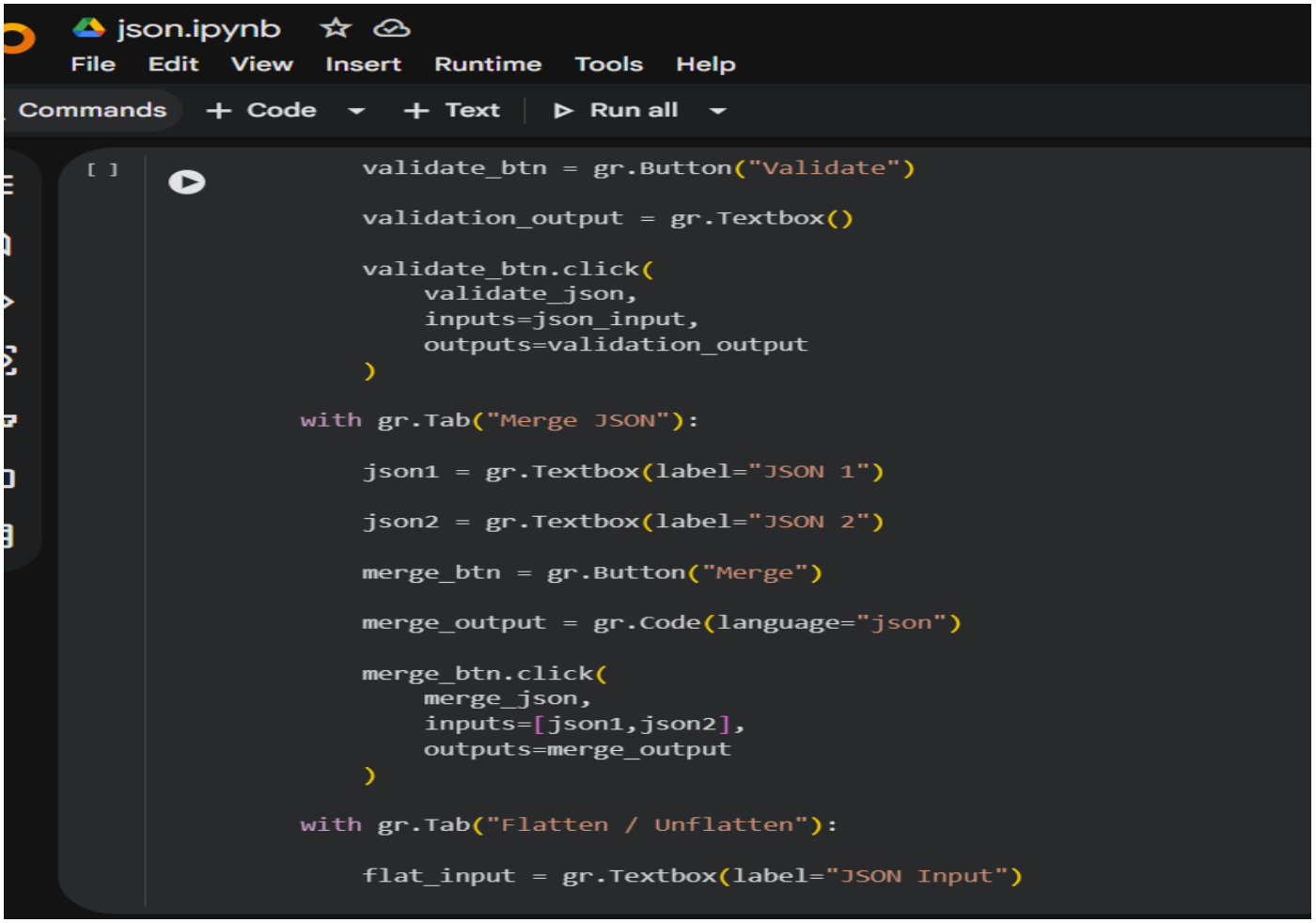
Activity 3.3: JSON Transformation Operations

Multiple transformation capabilities:

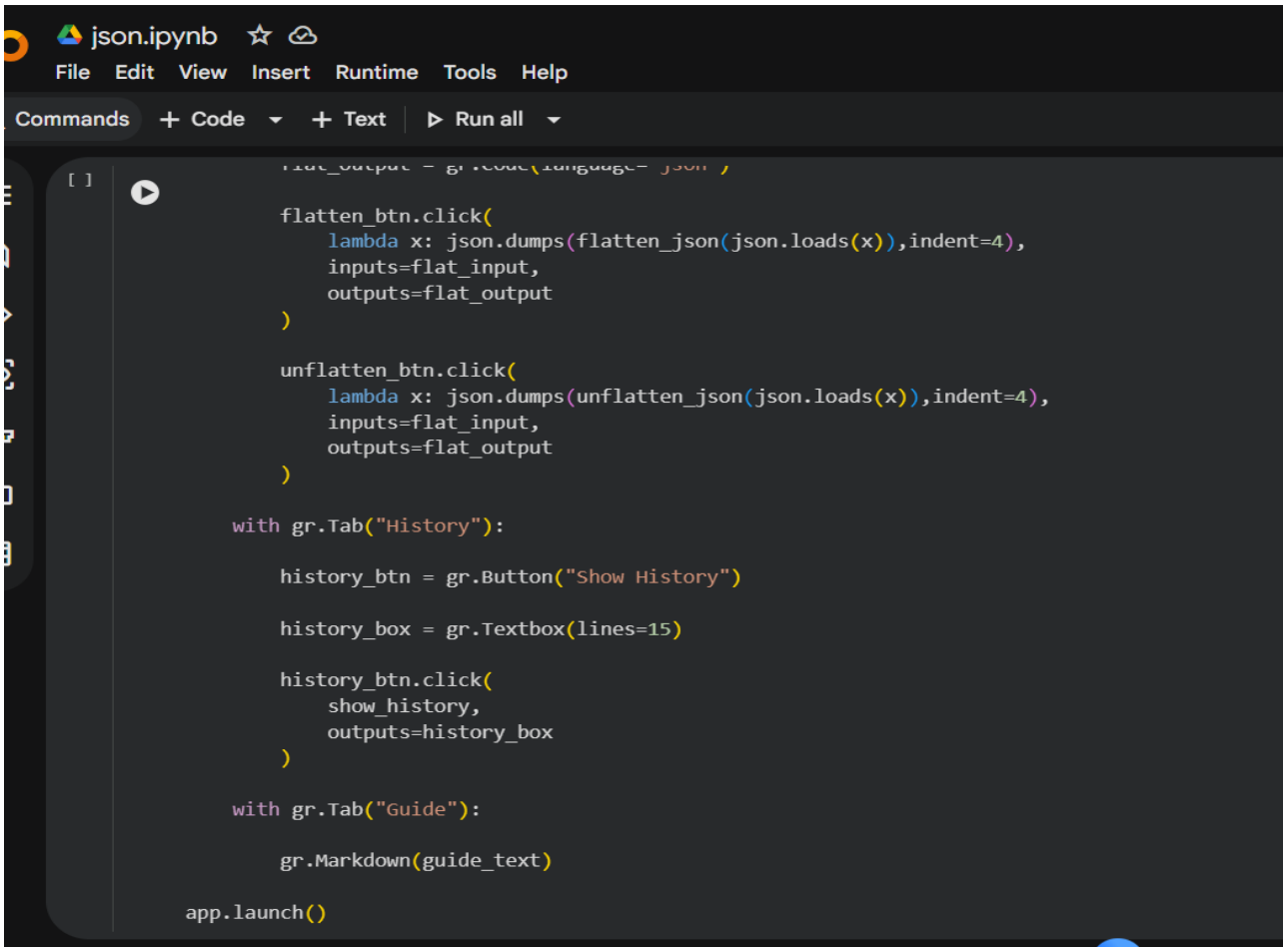
1. Flatten: Convert nested objects to dot-notation
   Input: {"user": {"name": "John", "age": 28}}
   Output: {"user.name": "John", "user.age": 28}

2. Unflatten: Reverse flattening
   Input: {"user.name": "John", "user.age": 28}
   Output: {"user": {"name": "John", "age": 28}}

3. Merge: Deep merge multiple JSON objects
4. Clean: Remove null/undefined values
5. Stats: Generate detailed statistics

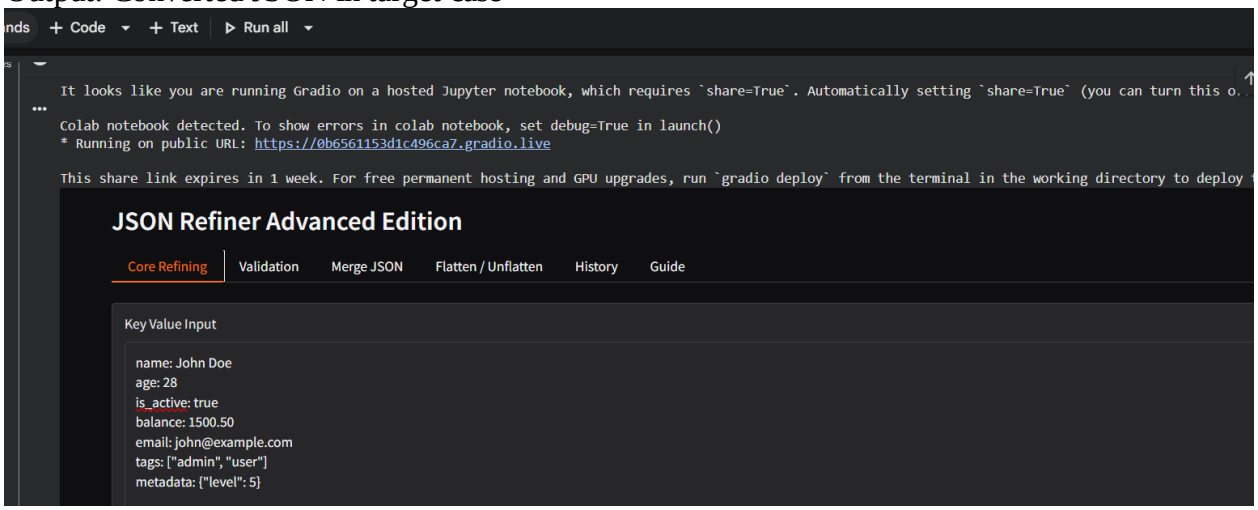



```
        validate_btn = gr.Button("Validate")

        validation_output = gr.Textbox()

        validate_btn.click(
            validate_json,
            inputs=json_input,
            outputs=validation_output
        )

    with gr.Tab("Merge JSON"):

        json1 = gr.Textbox(label="JSON 1")

        json2 = gr.Textbox(label="JSON 2")

        merge_btn = gr.Button("Merge")

        merge_output = gr.Code(language="json")

        merge_btn.click(
            merge_json,
            inputs=[json1,json2],
            outputs=merge_output
        )

    with gr.Tab("Flatten / Unflatten"):

        flat_input = gr.Textbox(label="JSON Input")
```

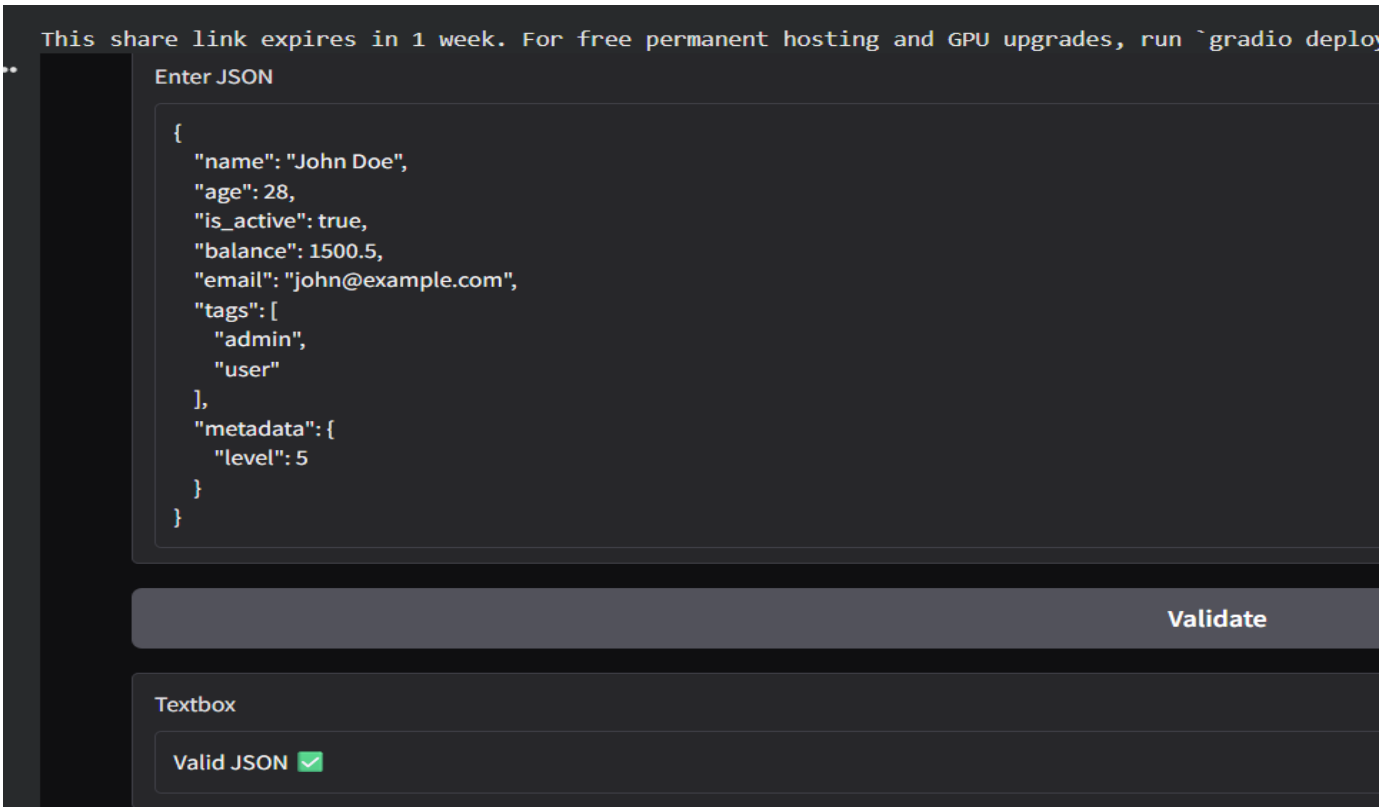


Output:

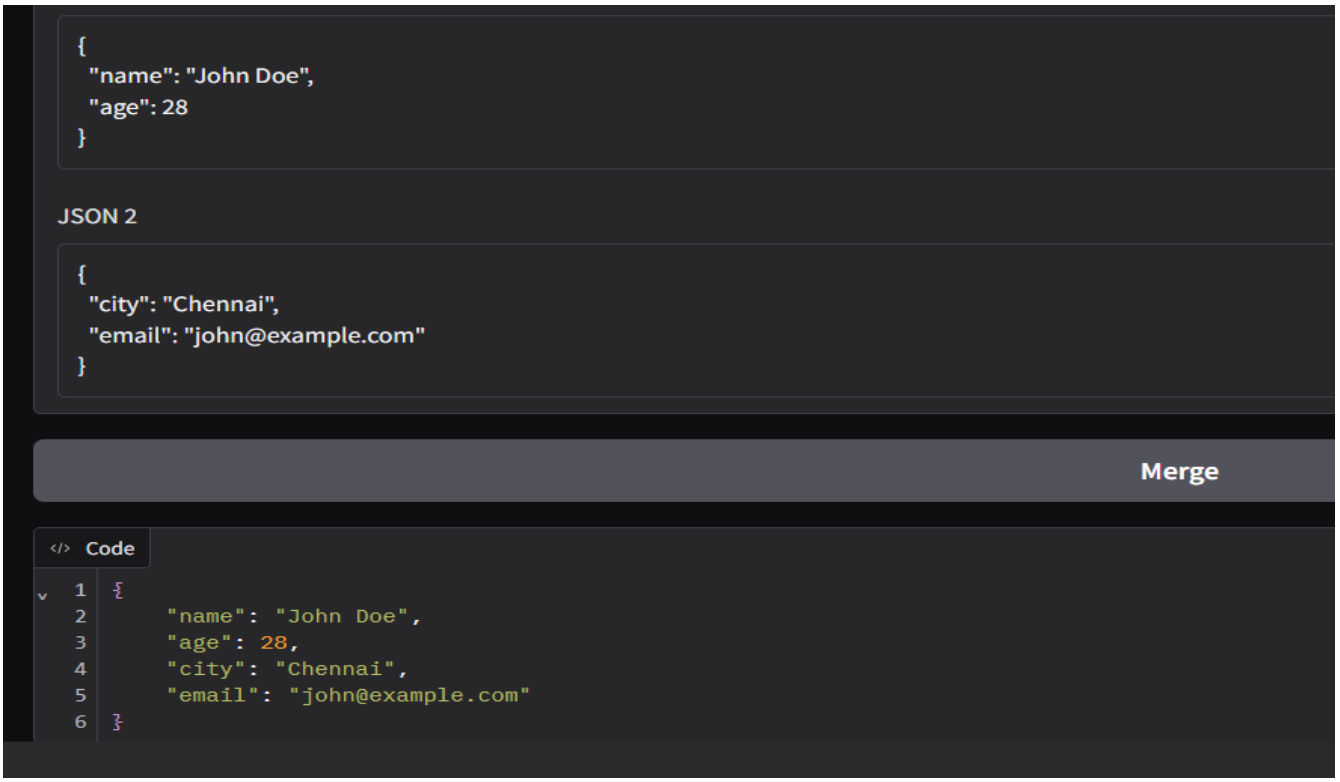
1. Case Conversion: Convert between case styles
   - Input: JSON
   - Output: Converted JSON in target case



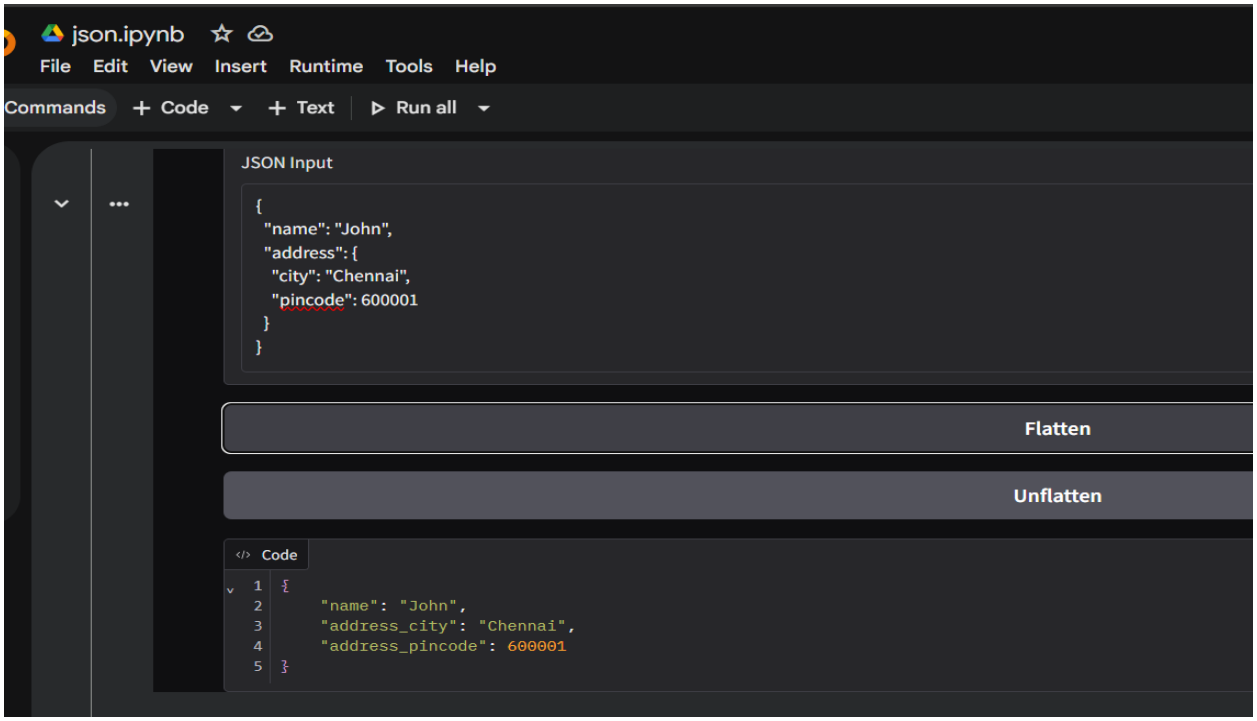
2.validation:



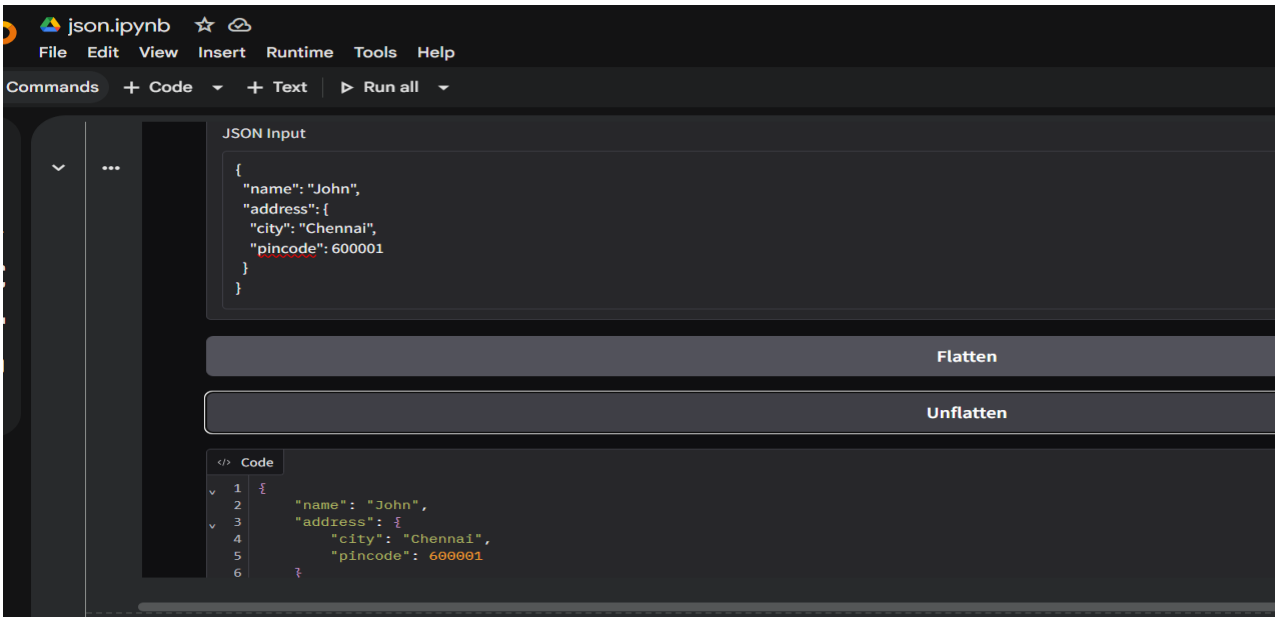
2. Merge JSON: Combine multiple objects
   - Input: Three JSON objects
   - Output: Merged result



3. Flatten/Unflatten: Transform nesting
   - Input: JSON
   - Output: Transformed structure

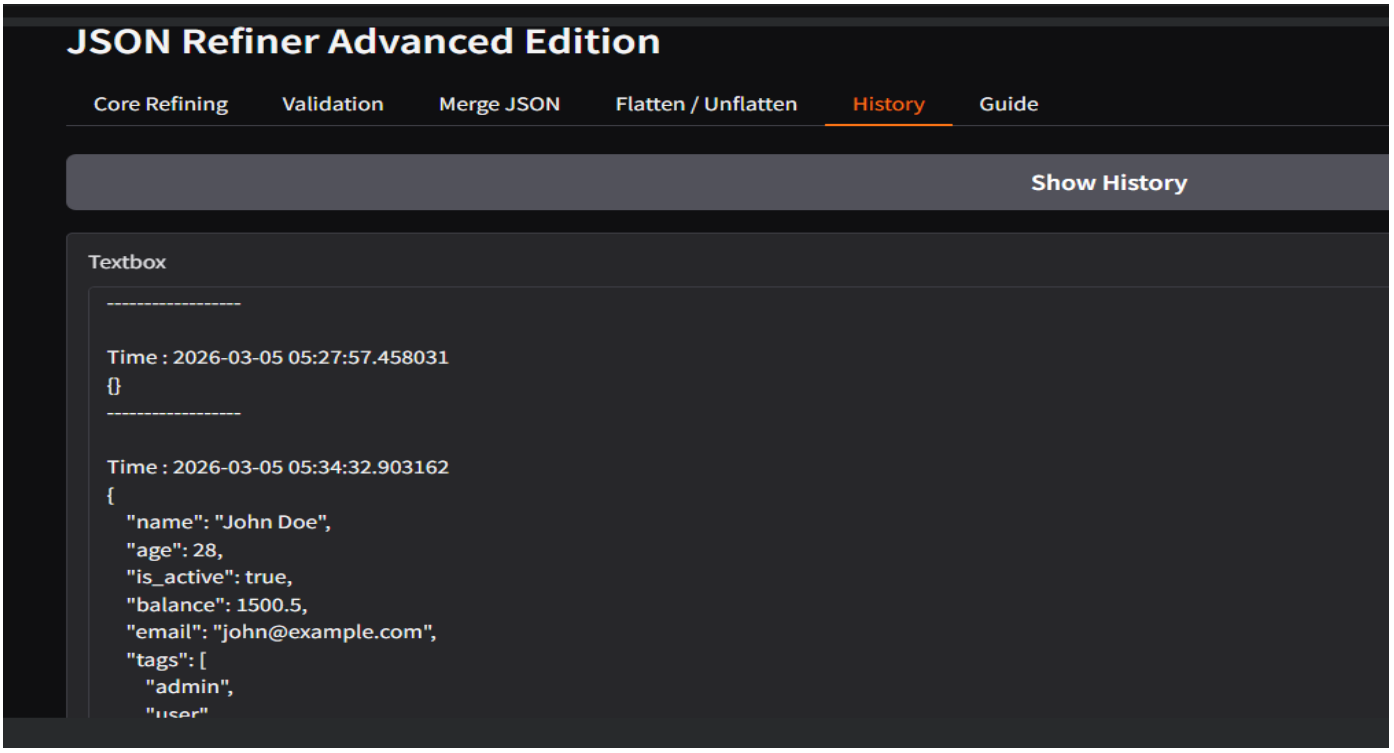


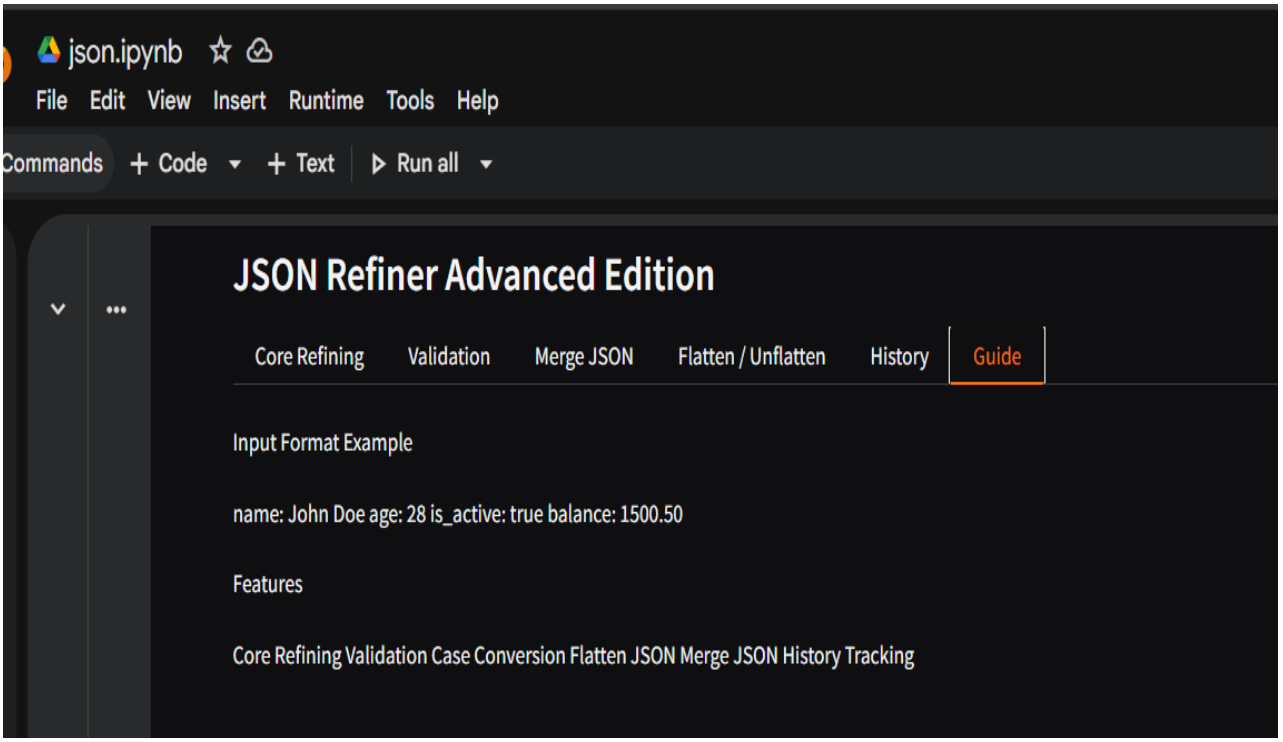
Output for UnFlatten:



4. **History: Track transformations**

- Download: Export history report
- Clear: Reset history





UI/UX Features

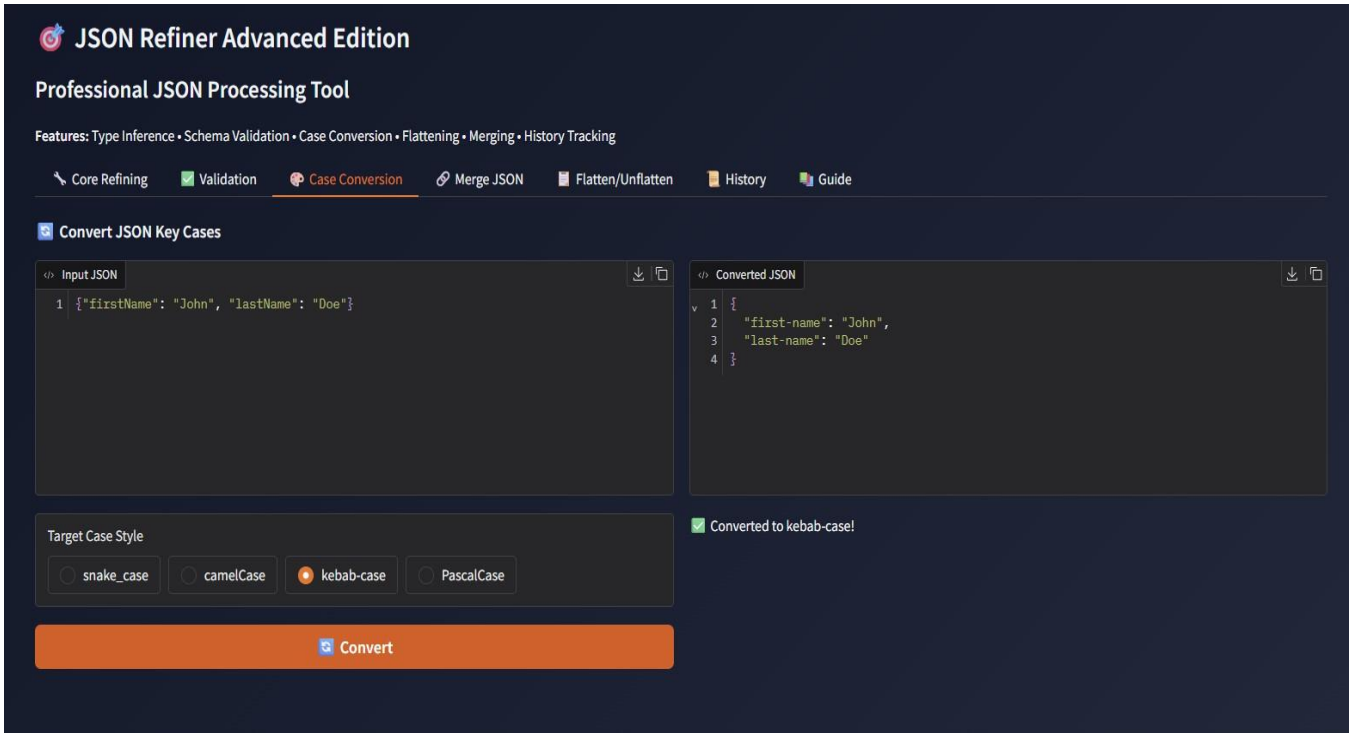
Professional interface design:

- Dark theme with gradient background
- Color-coded status messages (✓success, ✗error)
- Syntax-highlighted code blocks
- Real-time processing with instant feedback
- Detailed error reporting with line numbers
- Responsive layout for various screen sizes
- Intuitive button-based navigation

Activity 5.2: Functional Testing

Test Case 1: Basic Key-Value Conversion

Input:
name: John Doe
age: 28
active: true
alance: 1500.50



Conclusion

JSON Refiner Advanced Edition successfully delivers an intelligent JSON processing assistant that combines intuitive data transformation with powerful Python processing capabilities. By leveraging Gradio's flexible interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable JSON processing accessible to data analysts, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted data transformation, making professional JSON processing accessible while maintaining high data quality standards, comprehensive validation, and production-ready code output. With automatic type inference, four case conversion standards, advanced transformation operations, comprehensive validation capabilities, and detailed history tracking, JSON Refiner Advanced empowers users to process complex JSON data efficiently and accurately.

Future Enhancements:

The modular architecture allows for easy extension with additional features such as custom transformation rules, regex-based data extraction, integration with popular data tools (Pandas, NumPy), batch processing capabilities, API endpoint generation, custom validation rule creation, and template-based transformations. The system can be extended to support additional case styles, implement machine learning-based type inference, add data anonymization capabilities, and integrate with cloud storage services for large-scale data processing.

With continued development, JSON Refiner Advanced can evolve into a comprehensive data processing platform supporting diverse data formats while maintaining its focus on simplicity, accuracy, and user experience.